

CS 361 - Homework 02 - Theory

Professor Mark W. Boady

1 Overview

This homework consists of a few theory questions. The focus is on possible scenarios of execution for atomic statements.

All answers must be **typed** and submitted as a single **PDF**. The PDF must include your name and all questions must be clearly labeled. You do not have to repeat the question text, only label the questions clearly.

For all algorithm examples, lines that are not given numbers are considered starting conditions and not part of the algorithm. For example, if an algorithm has a local variable that starts at value $x = 10$, this might not be given a line number. It is a started condition for the process.

All lines are assumed to be **atomic statements**. Execution will never context switch in the middle of completing a numbered line.

When a question says **Explain** that means you may give a essay style answer, you do not need a full proof.

2 Questions

This homework is worth 40 points.

Question 1 : 12 points

Consider the following algorithm:

Algorithm Q1: Concurrent Locks	
Setup: Lock lk	
Thread P	Thread Q
p1: lk.lock() p2: print("P") p3: lk.unlock()	q1: lk.lock() q2: print("Q") q3: lk.unlock()

- (a) (6 points) Explain what possible outputs this algorithm can print?

Answer: The possible outputs of this algorithm is "PQ" or "QP" because p1 or q1 can take the lock first and print it letter first.

- (b) (6 points) Explain what can would if we removed the line p3: lk.unlock(). (make sure to state everything that can happen, not just one interleaving)

Answer: if we take p3 out of the code p1 lock will never be released and therefore, Q will never be print if P take the lock first. If thread Q takes the lock first, Q1 will print "QP" and lock will never release

Question 2 : 12 points

Consider the following algorithm:

Algorithm Q2: Looping Locks	
Setup: Lock lk and Boolean B = False	
Thread P	Thread Q
p1: lk.lock() p2: B = True p3: lk.unlock()	q1: lk.lock() q2: while not B q3: print("*") q4: lk.unlock()

- (a) (6 points) What is the minimum number of "*" that can be printed? Explain Why.

Answer the minimum number of "*" that can be printed is 0 because if Thread P take the lock first, B will definitely be true before Q can take the lock and run.

- (b) (6 points) What is the maximum number of "*" that can be printed? Explain Why. Answer: The maximum can be indefinitely because thread P technically can never take over the lock.

Question 3 : 16 points

We want to use a lock for something different. In this question, we will use it to send a **signal**.

You will start with the following algorithm that does not work correctly.

Algorithm Q3: Signaling	
Setup: Integer n = 0	
Thread P	Thread Q
p1: for i from 1 to 20 do p2: temp = n p3: n = temp + 1	q1: print(n)

Using **one** lock ensure that line q1 always happens **after** Thread P exits. Thread P needs to somehow **signal** Thread Q when it needs to run.

Rules:

- You may add **one** lock, but no other variables
- The code will be functionally sequential when you are done. (`print(n)` will run when P finishes)

Hint: You can call `lk.lock()` in the setup phase before the threads start.

Algorithm Q3: Signaling	
Setup: Integer <code>n = 0</code> , <code>lk.lock()</code>	
Thread P	Thread Q
p1: for i from 1 to 20 do p2: <code>temp = n</code> p3: <code>n = temp + 1</code> <code>lk.unlock()</code>	<code>lk.lock()</code> q1: <code>print(n)</code> <code>lk.unlock()</code>